

Angular Avançado · Módulo 7

Sessão 11

Signal Forms: o FlightEdit completo

João Gonçalves · joaogoncalves.net · Making the complicated simple

Onde ficámos na Sessão 10

Guards, interceptors e um form que já carrega e valida

Guards

- `canMatch` a `false` → a rota seguinte (`BoardingLocked`)
- `exitGuard`: `canDeactivate` + `DirtyAware`

Interceptors

- `authInterceptor`: o token, só para a nossa API
- `loggingInterceptor`: a ordem do array é a cadeia

FlightEdit (F1–F2)

- `findByIdResource` + `linkedSignal` + `form()`
- Validators inline · erros com `| json`
- `isDirty()` = `dirty()` do form

Falta: as **mensagens**, **gravar** e arrumar os validators.

Agenda de hoje

01	**Signal Forms (fim)**	Messages · submit · validation errors · conditional validation	≈ 110 min
02	**Resumo**	O FlightEdit completo · e o que vem na Sessão 12	≈ 5 min



Bloco A · Signal Forms

Signal Forms: o resto

Messages, submit e os validation errors num componente — no FlightEdit

O que falta no FlightEdit

F3 · Schema

- `schema<Flight>(…)` em `data/`
- `{ message }` em cada validator
- Sai o `| json` do F2

F4 · Gravar

- `submission: { action, onInvalid }`
- `<form [formRoot]>` + `<button>`
- Server error = mais um erro

F5 · Validation errors

- `[errors]` → as mensagens
- Vive em `shared/ui-forms`

F3 e F4: a teoria já foi dada na Sessão 10. O F5 é a ponte para a **arquitetura**.

Validators que dependem de outros fields

```
// schema logic: só aparece num voo atrasado
hidden(path.delay, { when: ({ valueOf }) => !valueOf(path.delayed) });

// estes validators só valem quando delayed é true
applyWhenValue(path, (flight) => flight.delayed, delayedFlight);

// um custom validator, no nível do voo
validate(path, ({ value }) =>
  value().from === value().to
    ? { kind: 'roundtrip', message: '…' } : null);
```

Notar

- `hidden` só marca o state: quem esconde é o `@if`
- `valueOf()` lê outro field (reativo)
- Validator no `path` raiz → erro em `flightForm().errors()`

No Angular 21: `hidden(path, fn)` — ainda funciona, mas está **deprecated** no 22.



Da teoria ao código

Agora, vamos escrever o código

- Mensagens e um schema num ficheiro próprio · gravar (`PUT`, pelos interceptors)
- Os validation errors num componente em `shared/ui-forms`
- O `delay` condicional e o validator From ≠ To

data/flight-schema.ts · data/flight-client.ts · flight-edit/* · shared/ui-forms/*

F3 · O schema num ficheiro próprio + message

domains/ticketing/data/flight-schema.ts

```ts
export const flightSchema = schema<Flight>((path) => {
  required(path.from, { message: 'From is required' });
  minLength(path.from, 3, { message: 'From needs at least 3 characters' });
  required(path.to, { message: 'To is required' });
  required(path.date, { message: 'Date is required' });
});
```

```ts
// flight-edit.ts
protected readonly flightForm =
  form(this.flight, flightSchema);
```

```html
<!-- por baixo do From, do To e da Date -->
@for (error of flightForm.from().errors(); track $index) {
  <div class="validation-errors">{{ error.message }}</div>
}
```

Por field: `flightForm.to().errors()` · todos de uma vez: `flightForm().errorSummary()`. O `@for` repetido vira um componente no F5.

F4 · Submit: gravar

```ts
// flight-client.ts
save(flight: Flight): Promise<Flight> {
  return firstValueFrom(this.http.put<Flight>(`${this.config.apiUrl}/${flight.id}`, flight));
}
```

```ts
feature-booking/flight-edit/flight-edit.ts
protected readonly flightForm = form(this.flight, flightSchema, {
  submission: {
    action: () => this.save(),
    onInvalid: (flightForm) => flightForm().errorSummary()[0]?.fieldTree().focusBoundControl(),
  },
});

private async save(): Promise<TreeValidationResult> {
  try {
    const flight = this.flight();
    await this.flightClient.save({ ...flight, date: new Date(flight.date).toISOString() });
    this.flightForm().reset();          // volta a pristine: o exitGuard deixa sair
    this.message.set('Saved');
    return null;
  } catch {
    return { kind: 'server', message: 'Could not save the flight' };
  }
}
```

F4 · Submit e experimentar

```
<form [formRoot]="flightForm">
  <fieldset> … </fieldset>

  <!-- erros do form (ex.: do servidor) -->
  @for (error of flightForm().errors(); track $index) {
    <div class="validation-errors">{{ error.message }}</div>
  }

  <button>Save</button>
  <small>{{ message() }}</small>
</form>
```

Experimentar

- From vazio → **Save** → foco no From, nada enviado
- "Funchal" → **Save** → `PUT` com token · "Saved"
- **Back to search** → sai sem perguntar
- URL errado (`${flight.id}x`) → 404 → "Could not save the flight"

O `PUT` passa pelos interceptors; o `reset()` deixa o `exitGuard` sossegado. Guards, interceptors e forms no mesmo ecrã.

A data: duas formas do mesmo instante

API (UTC): `08:15:00.000Z` → ao carregar (F1): `toLocalInput()` → **Input** (local): `09:15`

```
// ao gravar (F4): de volta a UTC
{ ...flight, date: new Date(flight.date).toISOString() }
//   cópia do voo  └ sem Z → hora local  └ escreve sempre em UTC
```

Porque não é automático?

O `datetime-local` **não tem fuso**: "local de quem?" — o Angular não decide por nós.

Alternativas

`fromLocalInput()` com nome · `transformedValue` num controlo à medida

F5 · Os validation errors num componente

shared/ui-forms/validation-errors/validation-pane.ts

```
@Component({
  selector: 'app-validation-pane',
  templateUrl: './validation-pane.html',
})
export class ValidationPane {
  readonly errors = input.required<ValidationError[]>();

  protected readonly messages = computed(() =>
    this.errors().map((error) => error.message ?? toMessage(error)));
}

// toMessage: 'required' → 'Enter a value' · 'min' → … · senão o kind

<!-- validation-pane.html -->
@for (message of messages(); track $index) {
  <div class="validation-errors">{{ message }}</div>
}
```

Porquê aqui?

- **ui**: só apresenta — `input`, sem requests nem state
- **shared**: não sabe nada de voos
- Mensagem por omissão (i18n entraria aqui)

F5 · Um componente, quatro sítios

```
<!-- antes: um @for por field -->
@for (error of flightForm.from().errors(); track $index) {
  <div class="validation-errors">{{ error.message }}</div>
}

<!-- depois -->
<app-validation-pane [errors]="flightForm.from().errors()" />
<app-validation-pane [errors]="flightForm.to().errors()" />
<app-validation-pane [errors]="flightForm.date().errors()" />

<!-- depois do </fieldset>: os erros do form -->
<app-validation-pane [errors]="flightForm().errors()" />
```

O ecrã fica **igual**; o código fica num só sítio — e num sítio que diz o que a peça é.

F6 · Conditional validation: o delay só num voo atrasado

```ts
// flight-schema.ts
hidden(path.delay, { when: ({ valueOf }) => !valueOf(path.delayed) });
applyWhenValue(path, (flight) => flight.delayed, delayedFlight);
…
export const delayedFlight = schema<Flight>((path) => {
  min(path.delay, 15, { message: 'A delay is at least 15 minutes' });
});
```

```html
flight-edit.html

@if (!flightForm.delay().hidden()) {
  <input type="number" [formField]="flightForm.delay" />
  <app-validation-pane [errors]="flightForm.delay().errors()" />
}
```

Experimentar

- Delayed → aparece, com erro
- 20 → ok · desligar → some

`hidden` = apresentação · `applyWhenValue` = a domain rule.

F7 · Um custom validator: From ≠ To

domains/ticketing/data/flight-validators.ts

```ts
export function validateRoundTrip(path: SchemaPath<Flight>) {
  validate(path, ({ value }) => {
    const { from, to } = value();
    if (from && from.toLowerCase() === to.toLowerCase()) {
      return { kind: 'roundtrip', message: 'From and To must be different' };
    }
    return null;
  });
}
```

```ts
// no fim do flightSchema
validateRoundTrip(path);
```

No nível do voo

O erro aparece no pane do form (F5), sem mexer no template.

Uma função com o `path` = um validator **com nome** e reutilizável, na pasta `data`.

Signal Forms: o que fica

Modelo

- Um signal (`linkedSignal` sobre o resource)
- `form()` → `FieldTree`: field vs estado

Schema

- Validators + schema logic
- Num ficheiro, na pasta `data`
- `validate()` para as nossas

Submit

- `submission` + `[formRoot]`
- Server errors no form
- `reset()` → o `exitGuard` deixa sair

`flight-schema.ts` em `data` · o pane em `shared/ui-forms` — **porquê aí?** É a Sessão 12.

Resumo da sessão

O FlightEdit completo

Schema

- `schema<Flight>` em `data`
- `{ message }` em cada validator
- `hidden` · `applyWhenValue` · `validate()`

Submit

- `submission: { action, onInvalid }`
- `[formRoot]` · server error = mais um erro
- `reset()` → o `exitGuard` deixa sair

Validation errors

- `ValidationPane` em `shared/ui-forms`
- `errors()` por field · `errorSummary()` todos
- `message ?? toMessage()`

Porque é que cada ficheiro ficou **onde** ficou? → **Sessão 12**.



Na Sessão 12

Arquitetura

- Domains e layers: como organizar uma app que cresce
- As dependency rules — e o Sheriff a verificá-las no Skyboard
- Onde vive cada peça (e cada store)